Ant Colony Optimization

Travelling Salesman Problem:

```python
import numpy as np
import random

cities = np.array([
    [10, 20], [20, 30], [30, 10], [40, 20],
    [50, 40], [30, 50], [15, 45], [5, 35]
])

n = len(cities)
ants = n
iterations = 100
alpha = 0.1
beta = 2
rho = 0.1
q0 = 0.9

dist = np.zeros((n, n))

for i in range(n):
    for j in range(n):
        dist[i][j] = np.linalg.norm(cities[i] - cities[j])

heuristic = 1 / (dist + np.eye(n))
pheromone = np.ones((n, n))

best_tour = None
best_length = float('inf')

for _ in range(iterations):
    for ant in range(ants):
        start = random.randrange(n)
        tour = [start]

        while len(tour) < n:
```

```
        while len(tour) < n:
            current = tour[-1]
            unvisited = [i for i in range(n) if i not in tour]

            values = [
                pheromone[current][j] * heuristic[current][j] ** beta
                for j in unvisited
            ]

            if random.random() < q0:
                next_city = unvisited[np.argmax(values)]
            else:
                probabilities = np.array(values) / sum(values)
                next_city = random.choices(unvisited, weights=probabilities)[0]

            tour.append(next_city)

            i, j = current, next_city
            pheromone[i][j] = (1 - rho) * pheromone[i][j] + rho / (n * 100)
            pheromone[j][i] = pheromone[i][j]

        length = sum(dist[tour[i]][tour[(i + 1) % n]] for i in range(n))

        if length < best_length:
            best_length = length
            best_tour = tour.copy()

    for i in range(n):
        a = best_tour[i]
        b = best_tour[(i + 1) % n]
        pheromone[a][b] = (1 - alpha) * pheromone[a][b] + alpha / best_length
        pheromone[b][a] = pheromone[a][b]

print("Best Tour:", best_tour + [best_tour[0]])
print("Best Distance:", round(best_length, 2))
```

Output:

```
Best Tour: [5, 6, 7, 1, 0, 2, 3, 4, 5]
Best Distance: 141.13
```